Repaso para la defensa del Proyecto 3

Pochoco SoC (Espino RV32E) de RTL a GDSII con LibreLane y GF180MCU

Arquitectura de Computadores 2026-20 — Grupo 2

Cómo usar este documento.

- Está pensado para los tres integrantes y como apunte durante la presentación. Todos deben poder responder cualquier pregunta del informe, del RTL y del ensamblador.
- Las preguntas marcadas con ⋆ son las **esenciales**: si solo se alcanza a estudiar una parte, que sea esa.
- Las respuestas son cortas a propósito. Los números salen del informe y de las métricas de las corridas del repositorio.
- Sobre lo que no se hizo o quedó incompleto, la respuesta correcta es decirlo con claridad (ver la sección 12). Eso se evalúa mejor que inventar.
- No incluye comandos: se repasan por separado.

Índice

1. Torpedo: las cifras que hay que saber **2**

2. Contexto general y flujo **4**

3. El procesador, el SoC y el juego **5**

4. Punto 3.1: Del RTL de FPGA al RTL de ASIC **6**

5. Punto 3.2: Memoria, programa y ensamblador **6**

6. Punto 3.3: Síntesis y trazabilidad **8**

7. Punto 3.4: Timing **8**

8. Punto 3.5: Floorplan, alimentación y placement **10**

9. Punto 3.6: Signoff y layout **11**

10. Punto 3.7: Simulación post-síntesis **12**

11. Preguntas conceptuales de apoyo **13**

12. Limitaciones honestas: qué decir sin dudar **14**

13. Dónde mirar cada tema (para estudiar o consultar) **14**

14. Si te preguntan algo que no sabes **14**

1. Torpedo: las cifras que hay que saber

Corrida final (`final_opt`): $T = 100$ ns (10 MHz), $U = 40\,\%$, estrategia 1

Concepto	Valor
Área del die / del core	0,4416 mm^2 (441.580 μm^2) / 410.059 μm^2
Costo estimado (US$23.000 por mm^2)	**US$10.156**
Instancias de celdas estándar (sin relleno)	9.535 (incluye 2.642 *tap*)
Flip-flops	1.069
Utilización de instancias tras PnR	51,2 %
Setup WNS (corner lento `max_ss` / típico `nom_tt`)	+37,22 ns / +67,04 ns
Hold WNS (típico / lento / peor de los 9 corners)	+0,44 / +1,06 / +0,17 ns
TNS setup y hold	0
DRC Magic / LVS / DRC de ruteo / antenas	0 / 0 / 0 / 0
Violaciones de *slew* máx. / capacitancia máx.	2.426 / 7 (solo avisos, en `max_ss`)
Potencia estimada	17,8 mW (casi todo dinámico; fugas $\approx$ 2,4 μW)
Longitud de cable / vías	397.054 μm / 47.238
Árbol de reloj (CTS)	115 buffers + 26 inversores; *skew* de setup $\approx$ 0,34 ns
Caída de tensión (IR drop) peor caso	0,22 mV sobre 5 V
Relleno (*fill*)	12.838 celdas (200.123 μm^2)

Herramientas y versiones

LibreLane 3.0.17 (Docker) · PDK `gf180mcuD` · biblioteca `gf180mcu_fd_sc_mcu7t5v0` · OpenROAD (build 2026-02-17) · OpenSTA 2.7.0 · Yosys 0.62 · Magic 8.3 · Netgen 1.5.316 · Verilator 5.044 (imagen).

Memoria (3.2): estrategia 1 frente a 2

	Estrategia 1 (ROM)	Estrategia 2 (FF)
Uso real del juego	114 palabras de instrucciones, 0 de datos	
Celdas tras síntesis	5.567	21.603
Área de celdas (μm^2)	162.735	654.901
Flip-flops	1.069	4.719
Die (mm^2) / costo	0,436 / US$10.024	1,695 / US$38.985
Setup lento (ns) / $f_{\text{máx}}$	+34,87 / 15,4 MHz	−740,81 / 1,19 MHz
Estado	Cerró	**Incompleta** (paso 43)

(Ambas con el RTL base, antes del comparador de saltos.)

Síntesis (3.3)

Módulo	Área (μm^2)	%
`espino_register_file` (544 `dffq`)	73.656	45,3
`espino_alu`	10.006	6,1
`pochoco_cycle_counter` (32 `dffrnq`)	3.943	**2,4**
Resto	75.131	46,2

Cinco celdas de mayor área: `dffq_1` (870; 34,0 %), `mux2_2` (693; 14,0 %), `dffrnq_1` (196; 9,0 %), `nand2_1` (857; 5,8 %), `mux4_1` (128; 5,5 %).

Timing (3.4): barrido del reloj (RTL base)

T (ns)	Setup típico	Setup lento	TNS lento
100	+65,76	+34,87	0
80	+45,76	+14,87	0
70	+35,76	+4,87	0
60	+25,76	−5,13	−444,56
50	+16,09	−14,46	−6.568,72

Hold constante en todos los puntos (típico 0,37 ns; lento 0,94 ns; TNS 0). Menor período medido que cierra: **70 ns (14,3 MHz)**. Estimado por extrapolación desde 100 ns: 65,1 ns (15,4 MHz); con el cambio de RTL, 62,8 ns (15,9 MHz).

Cambio de RTL y constantes

Comparador de saltos dedicado: slack +34,87 → +37,22 ns (**+2,35 ns**), área de celdas +1,0 % (+71 celdas), die +1,3 %.
A 10 MHz ($N = t/T$): `WAIT_CYCLES` = 30.000.000; `TENTH_CYCLES` = 1.000.000; `DEBOUNCE_CYCLES` = 200.000; `BLINK_CYCLES` = 2.500.000.

Floorplan (3.5): barrido de utilización (RTL base)

U (%)	Die (mm^2)	Costo (US$)	DRC iter. 0	Cierra
30	0,576	13.245	0	Sí
40	0,436	10.024	177	Sí
50	0,3515	8.083	260	Sí
60	0,295	6.784	97	Sí

Experimento `final_slew` (adicional, no es la corrida final)

Excluye celdas `dly*` y `buf_1` del *resizer*: *slew* 2.426 → 443 (−82 %); setup lento +37,22 → +56,56 ns; área de celdas −6 %; capacitancia máx. 7 → 20; signoff igualmente limpio.

Simulación post-síntesis (3.7)

Netlist `sim_corto`: 5.644 celdas. Resultado: `JUEGO: OK`, 0 fallos, 66 de 66 eventos, **trazas RTL y netlist idénticas**. Constantes cortas en ambas: espera de 3 s = 20.000 ciclos; décima = 1.000; antirrebote = 200; parpadeo = 2.000.

2. Contexto general y flujo

⋆ ¿Qué entrega este proyecto y qué es un GDSII?

Se lleva el Pochoco SoC (procesador Espino RV32E, periféricos y el juego de reflejos) desde el RTL hasta un **GDSII**, el archivo de layout (polígonos por capa) que se envía a fabricar. El resultado es un **macro digital** hecho con celdas estándar, no un chip completo.

⋆ ¿Qué diferencia hay entre este macro y un chip completo?

Los puertos del macro son terminales lógicos. Un chip real necesitaría **pads de I/O** con protección ESD y buffers de potencia para manejar LEDs y displays, además del encapsulado y las pruebas.

⋆ ¿Cuáles eran los parámetros de la corrida final y cuál es el criterio de "corrida cerrada"?

$T = 100$ ns (10 MHz) y utilización de core $U = 40\,\%$. Cerrada significa: el flujo termina, DRC y LVS limpios, antenas corregidas o justificadas, y setup y hold con WNS ≥ 0 y TNS $= 0$.

⋆ ¿Qué es LibreLane y qué hace?

Es un flujo abierto de diseño digital que encadena herramientas: Yosys (síntesis), OpenROAD (floorplan, *placement*, reloj, ruteo, STA), Magic y KLayout (layout y DRC) y Netgen (LVS). Se ejecutó en Docker. Cada etapa es un *paso* numerado (76 en total) que deja logs y reportes.

⋆ Resuman el flujo de RTL a GDSII, etapa por etapa.

1. **Lint** (Verilator): revisa el RTL.
2. **Síntesis** (Yosys): traduce el RTL a celdas de la biblioteca (*netlist*).
3. **Floorplan**: fija el tamaño del die según el área sintetizada y la utilización; define filas.
4. ***Tap* y *endcap*** y **red de alimentación** (PDN).
5. **Ubicación de pines** y ***placement*** global y detallado.
6. **Árbol de reloj** (CTS) y reparación de timing/*slew* (*resizer*).
7. **Ruteo** global y detallado, con reparación de antenas.
8. **Relleno** (*fill*), extracción de parásitos y STA post-PnR.
9. **Signoff**: DRC (Magic), LVS (Netgen), XOR entre layouts, chequeos de timing, *slew* y capacitancia.
10. **Salida**: GDS, LEF, netlist.

¿Qué es el PDK y qué es la biblioteca de celdas?

El PDK (`gf180mcuD`) contiene las reglas de fabricación (DRC), los modelos y las capas del proceso GF180MCU. La biblioteca `gf180mcu_fd_sc_mcu7t5v0` es el conjunto de celdas estándar (compuertas, flip-flops, buffers) de 7 pistas a 5 V con las que se arma el diseño.

¿Por qué es importante registrar las versiones?

Para que el resultado sea **reproducible**: otra persona con las mismas versiones de LibreLane, PDK y herramientas debe obtener lo mismo. Están en el README y en `versiones_herramientas.txt`.

¿Cuánto demora una corrida y con qué recursos se hicieron?

Entre 5,3 y 7,0 minutos por corrida de la estrategia 1 (ruteo detallado 1,3–1,8 min). Notebook con Windows, WSL2 y Docker, CPU Intel i5-12450H (8 núcleos, 12 hilos), 32 GB de RAM (WSL2 ve 15 GiB). El pico fue de 945 MiB de RAM, así que la memoria no fue un límite.

3. El procesador, el SoC y el juego

⋆ ¿Qué es el Espino y qué características tiene?

Un núcleo **RISC-V RV32E** (16 registros de 32 bits, `x0` fijo en cero) de **dos etapas**: fetch (`espino_if_stage`) y decodificación/ejecución (`espino_id_stage`, que contiene la ALU y la resolución de saltos). No tiene multiplicación ni división, y los *shifts* están deshabilitados (se ejecutan como suma), por eso el juego duplica con `add x,x,x` y divide con restas sucesivas.

⋆ ¿Qué es el Pochoco SoC y cómo se comunica el juego con el hardware?

Es el SoC del curso: el núcleo Espino más memoria, periféricos (LEDs, botones, displays de 7 segmentos, SPI) y un contador de ciclos. Los periféricos están **mapeados en memoria** desde `0x80000000`: displays en el offset 0, LEDs en el 4, botones en el 8 y el contador de ciclos en `0x8000000C`. El juego fija esa base en el registro `s0`.

¿Para qué sirve el contador de ciclos?

Mide el tiempo de reacción en ciclos de reloj; fue lo que se agregó en el Proyecto 2. Aporta solo el 2,4 % del área sintetizada (32 flip-flops con reset).

⋆ ¿Cómo funciona el juego?

Los LEDs parpadean mientras se espera. En cada ronda se encienden los 4 LEDs durante 3 s y luego queda uno solo, el objetivo. El jugador aprieta el botón de ese LED y los displays muestran el tiempo en décimas de segundo. Un botón equivocado muestra `EE` y la ronda se repite, conservando las correctas. Tras 10 respuestas correctas se muestra el último tiempo y luego el promedio.

¿Por qué la lectura del register file es síncrona y qué consecuencia tiene?

Porque en la FPGA se hacía con bloques BRAM, que leen con un ciclo de latencia. Se mantuvo ese comportamiento por ciclos en la versión ASIC para no tocar el núcleo: el dato aparece un ciclo después de pedir la dirección, y por eso el núcleo espera un ciclo (`ops_valid_q`).

¿Por qué `x0` no se almacena en el register file?

Porque siempre vale cero: se lee como cero con una comparación de la dirección. Así se ahorran 32 flip-flops. Por eso el banco guarda 15 registros (`x1` a `x15`).

4. Punto 3.1: Del RTL de FPGA al RTL de ASIC

⋆ ¿Qué hace `asic_top` y por qué se creó?

Es el módulo superior del macro: instancia el SoC (con 8 palabras de datos) **sin elementos de FPGA**. Sus puertos son las funciones del juego: reloj, reset, 4 LEDs, 4 botones, 2 displays de 7 segmentos (7 bits c/u) y SPI. Además agrega un sincronizador de reset y sincronizadores para los botones.

⋆ ¿Cuáles son las construcciones de FPGA que se identificaron y cómo se reemplazó cada una?

- **BRAM `SB_RAM40_4K`** (register file): es una primitiva de la iCE40 que no existe en el PDK. Evidencia: error `MODMISSING` en el lint y el flujo se detiene en el paso 5. Se reemplazó por un register file de flip-flops.
- **Contador de reset al encender** (`por_cnt = 12'b0`): dependía del valor inicial de los flip-flops de la FPGA. Evidencia: aviso `PROCASSINIT`. Se reemplazó por reset externo con sincronizador.
- **`initial` + `$readmemh`** (RAM): **no dio ningún aviso**. Se reemplazó por una ROM generada por el ensamblador y RAM de datos con reset.
- **Constantes del reloj de 25 MHz**: tampoco dio aviso. Se recalcularon para 10 MHz.

⋆ ¿Por qué `$readmemh` sin ningún aviso sigue siendo un problema en ASIC?

Porque Yosys lo trata como valor inicial de la memoria, pero un chip **no carga ningún archivo al encenderse**: sus bits arrancan en un estado indeterminado. El programa del juego comenzaría con basura. Por eso el programa debe estar en la lógica (ROM) o en celdas con reset.

⋆ ¿Por qué un flip-flop sin reset no basta en ASIC, y qué hace el sincronizador de reset?

En FPGA los flip-flops arrancan en un valor conocido al configurarse; en silicio arrancan aleatorios. El reset se aplica de forma **asíncrona** (el sistema se limpia de inmediato) y se libera de forma **síncrona** con dos flip-flops, para que todo el sistema salga del reset en el mismo flanco de reloj y evitar violaciones de *recovery/removal*.

¿Por qué se usan dos flip-flops para sincronizar los botones?

Los botones son entradas asíncronas al reloj. Un solo flip-flop podría quedar en **metaestabilidad**; el segundo da tiempo a que se resuelva antes de que el resto del diseño use la señal.

¿Qué efecto tiene no corregir el RTL de FPGA?

Con un modelo de BRAM para llegar a la síntesis (`run0b`), el diseño sale con 68.512 celdas, 17.233 flip-flops y unos 2,33 mm^2: más de 5 veces el área del diseño final.

¿Qué quiere decir la evidencia "log y línea"?

El enunciado pide mostrar el aviso de LibreLane que reveló cada construcción. En el informe están los archivos de log y las líneas (por ejemplo el lint con `MODMISSING` y la síntesis con `PROCASSINIT`), de las corridas `run0` y `run0b`.

5. Punto 3.2: Memoria, programa y ensamblador

⋆ ¿Por qué no basta con `game.hex` en un ASIC?

Porque en silicio no hay un archivo que se cargue al encender. El programa tiene que ser parte del circuito. De ahí las dos estrategias.

⋆ Expliquen las dos estrategias de memoria.

1. **ROM combinacional + RAM de datos reducida**: el ensamblador genera un módulo Verilog con un `case` sobre la dirección que devuelve cada instrucción (con registro de salida para mantener la lectura síncrona), más una RAM de datos de 8 palabras con flip-flops. La síntesis la convierte en lógica.

2. **Memoria unificada de flip-flops**: 114 + 8 = 122 palabras, todas con flip-flops (y valores iniciales generados por el ensamblador).

⋆ ¿Cuántas palabras usa realmente el juego y cómo lo supieron?

114 palabras de instrucciones y 0 de datos, con el modo de reporte del ensamblador. Las 12 cargas y 6 escrituras del programa van todas a periféricos mediante `s0`; ninguna usa la RAM de datos. Se dejaron 8 palabras de datos para mantener una RAM funcional. Ambas cifras están bajo el máximo recomendado (256 y 64).

⋆ ¿Cuál estrategia es mejor y por qué?

La **estrategia 1**. Con la 2 el área de celdas se multiplica por 4 (654.901 frente a 162.735 μm^2), los flip-flops pasan de 1.069 a 4.719, el die de 0,436 a 1,695 mm^2 (costo US$10.024 frente a US$38.985) y el cable se vuelve casi 10 veces más largo, lo que hunde el timing. Una ROM como lógica se optimiza con funciones booleanas compartidas; una memoria de flip-flops necesita 32 por palabra más multiplexores de lectura y escritura.

⋆ ¿Qué extensiones se le hicieron al ensamblador?

Opciones para: generar la ROM en Verilog; generar la inicialización de la memoria de flip-flops; reportar palabras de instrucciones y de datos usadas; y definir constantes al ensamblar (para cambiar las esperas sin editar `game.s`). Se agregaron pruebas: son 25 en total y pasan todas.

¿Qué es un ensamblador y qué hace el de ustedes?

Traduce el programa en lenguaje ensamblador RISC-V (`game.s`) a código máquina de 32 bits por instrucción. El del grupo, `pochoasm.py`, además genera los archivos que el hardware necesita (ROM en Verilog, inicialización).

¿Qué pasó con la estrategia 2?

No se completó: la corrida se detuvo en el paso 43 (STA intermedio) con una red con violación de antena, y el ruteo detallado quedó estancado en una violación de DRC entre las iteraciones 44 y 60. Sus valores de timing son los de ese paso intermedio, indicativos y no un cierre. No se repitió por falta de tiempo.

¿Por qué conservar 8 palabras de RAM de datos si el juego usa 0?

Para que el SoC siga siendo funcional con otros programas que sí usen la pila o variables; el costo es pequeño (8 palabras de flip-flops).

6. Punto 3.3: Síntesis y trazabilidad

⋆ ¿Cuáles son las cinco celdas más usadas y qué las explica?

Por área: `dffq_1` (870; 34 %), flip-flops sin reset, sobre todo los 544 del register file; `mux2_2` (693; 14 %), lectura y escritura del register file y selección de operandos; `dffrnq_1` (196; 9 %), registros con reset (PC, contador de ciclos, periféricos); `nand2_1` (857; 5,8 %), lógica de la ALU, decodificación y ROM; `mux4_1` (128; 5,5 %), árbol de lectura del register file.

⋆ ¿Qué módulo aporta más área y qué fracción es el contador de ciclos?

El **register file**: 73.656 μm^2, 45,3 % del total. El **contador de ciclos** agregado en el Proyecto 2: 3.943 μm^2, **2,4 %**.

⋆ ¿Cómo conservaron los nombres para identificar el register file, la ALU y el contador de ciclos?

La síntesis se configuró en modo "aplanar" pero con una lista de módulos cuya jerarquía se conserva: `espino_register_file`, `espino_alu` y `pochoco_cycle_counter`. Así el reporte entrega áreas por módulo y los nombres de instancia llegan al layout (`u_soc.u_core.u_rf/`, `u_soc.u_core.u_id.u_alu/`, `u_soc.u_periph.u_cycle_counter`).

⋆ ¿Coincidió el resultado con lo esperado antes de sintetizar?

Sí. Desde el RTL se deduce que el register file tiene $15 \cdot 32 + 2 \cdot 32 = 544$ flip-flops (15 registros más los dos puertos de lectura registrados), y el reporte muestra exactamente 544. Con 63,7 μm^2 por flip-flop dan $\approx$34.650 μm^2; sumando multiplexores de escritura ($\approx$15.800) y de lectura ($\approx$22.500) da $\approx$72.950, frente a 73.656 del reporte (1 % de diferencia). Lo no intuitivo: el register file pesa más del doble que sus flip-flops por la lógica de selección.

¿Por qué hay flip-flops con reset (`dffrnq`) y sin reset (`dffq`)?

El register file no se reinicia (`rst_ni` no se usa); los registros de control (PC, contador de ciclos, estado de periféricos) sí necesitan un valor conocido tras el reset. La celda con reset es más grande.

¿Qué hace la síntesis lógica?

Convierte el RTL (Verilog) en una red de celdas de la biblioteca (el *netlist*), optimizando área; no conoce todavía la ubicación ni los cables.

7. Punto 3.4: Timing

⋆ Definan setup, hold, slack, WNS y TNS.

- **Setup**: el dato debe llegar al flip-flop destino *antes* del siguiente flanco (menos el tiempo de preparación). Limita la frecuencia.
- **Hold**: el dato debe *mantenerse* estable un tiempo después del flanco. No depende del período.
- **Slack**: tiempo requerido menos tiempo de llegada; positivo es bueno.
- **WNS**: el peor slack. **TNS**: suma de los slacks negativos (0 significa que no hay ninguno).

⋆ ¿Qué son los corners y por qué se pide el típico y el más lento?

Combinaciones de proceso, tensión y temperatura (el flujo analiza 9). El **típico** es `nom_tt_025C_5v00`; el **más lento** es `max_ss_125C_4v50` (transistores lentos, 125 °C, 4,5 V), que da los retardos mayores y por eso es el crítico para setup. Se exige cumplir en todos.

⋆ ¿Cómo hicieron el barrido y cuál es el menor período que cierra?

Cinco puntos de `CLOCK_PERIOD`: 50, 60, 70, 80 y 100 ns, con $U = 40$ %. Cierra hasta **70 ns (14,3 MHz)**; en 60 ns el corner lento falla ($-$5,13 ns) y en 50 ns también ($-$14,46 ns, TNS $-$6.568,72). Desde el slack de 100 ns se

estima un mínimo de 65,1 ns, y con el cambio de RTL de 62,8 ns (15,9 MHz), pero eso es extrapolación.

⋆ ¿Por qué el slack cambia exactamente 10 ns por cada 10 ns de período?

Porque el slack es el período menos el retardo del camino, y el retardo no cambió (misma ruta, mismo circuito). Por eso la frecuencia máxima se obtiene como $1/(T - \text{slack})$.

⋆ ¿Por qué el hold no cambia con el período?

Porque depende de los retardos mínimos entre flip-flops del mismo flanco, no del tiempo entre flancos. En la tabla es constante: 0,37 ns típico y 0,94 ns lento.

⋆ Describan la ruta crítica.

En la corrida `final`: inicio en `_5373_` (`dffq_1`, `u_soc.instr_rdata[3]`), fin en `_5228_` (`dffrnq_1`, `u_soc.instr_addr[23]`, el PC). 40 celdas y 67,19 ns de llegada en el corner lento. Función: **decodificación → ejecución (comparación en la ALU) → resolución del salto → próximo PC (*fetch*)**. De esos 67 ns, 36,6 son celdas de retardo `dlyb_1` y unos 14 son la ALU.

⋆ ¿Qué cambio al RTL propusieron y qué efecto tuvo?

Un **comparador de saltos dedicado** en la etapa de decodificación (igualdad, menor con y sin signo según el campo `funct3`), en paralelo con la ALU, para sacar la comparación de la ruta hacia el PC. Resultado: slack +34,87 → +37,22 ns (**+2,35 ns**), área de celdas +1,0 % (71 celdas más), die +1,3 %. La nueva ruta crítica pasa a ser decodificación → ALU → *write-back* (fin en el register file). La mejora es modesta porque la ruta sigue dominada por las celdas de retardo.

⋆ ¿Qué constantes de `game.s` cambian con $T = 100$ ns y cómo se calcula?

Con $T = 100$ ns, $f = 1/T = 10$ MHz, así que **1 s** $= 10^7$ **ciclos** y $N = t/T$. `WAIT_CYCLES` (3 s) = 30.000.000; `TENTH_CYCLES` (0,1 s) = 1.000.000; `DEBOUNCE_CYCLES` (20 ms) = 200.000; `BLINK_CYCLES` (0,25 s) = 2.500.000. A 25 MHz serían 75.000.000, 2.500.000, 500.000 y 6.250.000. El programa se regeneró con esos valores.

⋆ ¿Por qué las áreas y el buffering pueden cambiar con cada `CLOCK_PERIOD`?

Porque cada valor de reloj hace que el flujo vuelva a optimizar (dimensionamiento de celdas, buffers, *placement*). En el barrido el número de celdas fue idéntico entre 100 y 60 ns y solo cambió en 50 ns. Una explicación *probable, no verificada* es que el *resizer* repara solo en el corner típico, donde el slack siempre fue positivo.

¿Por qué cierra con tanta holgura si se pidió 100 ns?

Porque el camino más largo del diseño tarda unos 63 ns en el corner lento, bastante menos que 100 ns. El diseño es pequeño y su límite real está en la lógica del núcleo y en las celdas de retardo que insertó el *resizer*.

¿Qué son las celdas `dlyb_1` y por qué hay tantas?

Son celdas de retardo de la biblioteca. El *resizer* las usa como *buffers* al reparar *slew*, capacitancia y *fanout*, y hay unas 622 en el netlist final. Como tienen poca fuerza y mucho retardo, dominan la ruta crítica y explican las violaciones de *slew* en el corner lento.

8. Punto 3.5: Floorplan, alimentación y placement

⋆ ¿Cómo se define el tamaño del die?

Con `FP_CORE_UTIL`: el core se dimensiona como el área sintetizada dividida por la utilización. Con $U = 40\,\%$: 162.735/0,40 ≈ 406.800 μm^2, cercano al core de 410.059 μm^2. El die (441.580 μm^2) agrega el margen exterior. Tras *placement* y reparaciones, la utilización real sube a ≈51 % porque se agregan buffers, *tap* y árbol de reloj.

⋆ ¿Qué hallaron en el barrido de utilización?

Con $U = 30, 40, 50, 60\,\%$ todas las corridas completaron el flujo con DRC, LVS y antenas limpios. El área del die baja de 0,576 a 0,295 mm^2 y el costo de US$13.245 a US$6.784. Las violaciones de DRC al comienzo del ruteo no son monótonas (0, 177, 260 y 97), pero el ruteador las eliminó siempre. El área mínima que cierra entre los puntos probados es la de $U = 60\,\%$.

⋆ ¿Cómo calcularon el costo y qué limitaciones tiene?

0,4416 mm^2 × US$23.000/mm^2 ≈ US$10.156. Limitaciones: el costo real no es lineal con el área (el *yield* cae al crecer el área, hay costos fijos de máscaras y corridas multiproyecto con tamaño mínimo, y el encapsulado, las pruebas y los pads no están incluidos), y se usó el área del die, no la de las celdas.

⋆ ¿Qué decisión tuvo mayor impacto en área y costo?

La **estrategia de memoria**: pasar de ROM a flip-flops multiplicó el die por 3,9 (0,436 a 1,695 mm^2) y el costo de US$10.024 a US$38.985. Variar la utilización de 30 a 60 % cambió el costo en un factor de 2,0, y el cambio de RTL del timing en 1,3 %.

⋆ ¿Por qué pusieron cada grupo de pines en ese borde?

Por función: **oeste**, reloj y reset (control global, un solo lado para el árbol de reloj); **sur**, botones y LEDs (la interacción con el jugador); **norte**, los 14 pines de los displays en un grupo contiguo; **este**, el SPI, lejos del reloj. Agrupar acorta cables y reduce cruces.

¿Qué es la red de alimentación (PDN) y qué se midió de ella?

La estructura de cables que lleva VDD y VSS a todas las celdas, conectada a los rieles de cada fila. Se generó en el paso *generatepdn*. La caída de tensión peor caso (IR drop) fue de apenas 0,22 mV sobre 5 V, así que la red es más que suficiente.

¿Qué son las celdas *tap*, *endcap* y *fill*?

Tap: celdas que conectan los pozos al sustrato para evitar el *latch-up* (2.642 en este diseño). *Endcap*: celdas al inicio y final de cada fila (326). *Fill*: relleno de los espacios vacíos para cumplir reglas de densidad y continuidad de los pozos (12.838 celdas, 200.123 μm^2); por eso la utilización de instancias se reporta sin ellas.

¿Qué es el *placement* y el árbol de reloj?

Placement global: reparte las celdas minimizando la longitud de cable; el detallado las acomoda legalmente en filas. CTS (*clock tree synthesis*): construye la red que reparte el reloj a los 1.069 flip-flops con poca diferencia de llegada (*skew*); aquí usó 115 buffers y 26 inversores y el *skew* de setup fue de unos 0,34 ns.

¿Por qué no probaron 70 % de utilización?

Se exigían al menos tres valores y se probaron cuatro (30, 40, 50 y 60 %). Con más utilización el ruteo se vuelve más difícil y las corridas se pueden alargar mucho; por tiempo no se exploró 70 %.

9. Punto 3.6: Signoff y layout

⋆ ¿Qué son DRC, LVS y antena, y qué resultado obtuvieron?

- **DRC**: el layout cumple las reglas geométricas de fabricación. Se usó **Magic**; resultado **0**. El DRC de KLayout se omite para `gf180mcuD` porque el PDK no trae su archivo de reglas para esa herramienta.
- **LVS**: el layout implementa exactamente el mismo circuito que el netlist. Se usó **Netgen**; resultado **0 diferencias**.
- **Antena**: control de carga acumulada durante la fabricación. Resultado final: **0 redes con violación**.

Además: 0 violaciones de DRC de ruteo y diferencia 0 en el XOR entre los GDS de Magic y KLayout.

⋆ ¿Qué es una violación de antena y cómo se repara?

Durante la fabricación, un cable largo de metal conectado solo a la compuerta de un transistor acumula carga del plasma y puede dañar el óxido. Se repara con ***jumpers*** (cambiar el tramo largo a otra capa de metal, de modo que la compuerta quede conectada por un tramo corto en cada capa) y con **diodos de antena** que derivan la carga al sustrato.

⋆ Describan una violación que apareció en el desarrollo, dónde estaba y cómo se resolvió.

Tras el ruteo global (paso 40) hubo **14 redes con violación de antena**, por ejemplo `rf_wdata[1]`, con receptores en los buffers de *fanout* del register file (`u_soc.u_core.u_rf/fanout80/I`, `fanout81`, `fanout74`), en la zona del register file ($y \approx 314$–651 μm). El paso 42 las reparó: 17 violaciones → 18 *jumpers* en 16 redes, luego 1 diodo y 8 *jumpers* más, hasta 0. El reporte del paso 46 quedó vacío y el layout final tiene 8 instancias `ANTENNA_*` (por ejemplo `ANTENNA_8` en (57,7; 541,0) μm).

⋆ ¿Qué pasó con el corto en Metal2 que mencionó el ayudante?

En el ruteo detallado (paso 44) las violaciones de DRC bajaron 2.004 → 669 → 545 → **1** → 0. Esa violación era un **corto (*short*) en Metal2**. En nuestra corrida desapareció en la iteración siguiente. Luego el chequeo de antenas del mismo paso encontró 5 redes, se hizo una iteración de reparación (DRC 100, 16, 13 y 0) y la verificación posterior dio 0. El signoff independiente (Magic y Netgen) quedó limpio. En la corrida del ayudante ese corto sí quedó hasta el final de la reparación de antenas, pero su signoff también dio 0, por lo que el resultado era válido.

El ayudante probó una opción distinta para el ruteo. ¿Qué logró y por qué no la usaron?

Limitó las iteraciones de reparación de antenas a 1: el ruteo llegó a 0 violaciones, pero le quedaron 9 de antena por corregir o justificar. En nuestro caso ya teníamos antenas en 0 y DRC limpio, así que cambiarlo habría cambiado un problema resuelto por uno nuevo.

⋆ ¿Cómo marcaron el register file, la ALU y el contador de ciclos en el layout?

Con los nombres jerárquicos conservados en la síntesis, ubicando las celdas por prefijo de instancia. Extensión aproximada (en μm): register file x 23–635, y 314–651 (1.845 celdas); ALU x 207–488, y 318–439 (580); contador de ciclos x 394–463, y 20–184 (114). Hay una captura anotada en KLayout y otra con las celdas por módulo.

¿Qué son las violaciones de *slew* y capacitancia máxima? ¿Importan?

Son avisos de que algunas señales cambian demasiado lento (*slew*) o que un nodo carga más de lo que su celda soporta. Aparecen en el corner lento (125 °C, 4,5 V), que degrada los flancos, y se deben sobre todo a las celdas de retardo débiles. No eran criterio de cierre del enunciado, pero reducirlas mejora la calidad del diseño.

¿Qué se hizo con el experimento `final_slew`?

Una corrida adicional (mismo RTL, $T = 100$ ns, $U = 40\,\%$) que excluye las celdas `dly*` y `buf_1` del *resizer* y exige más margen de slack. Resultado: *slew* de 2.426 a 443 ($-82\,\%$), setup lento de +37,22 a +56,56 ns, área de celdas $-6\,\%$, signoff limpio; a cambio, capacitancia máxima de 7 a 20. No se usó para el resto del informe porque layout, síntesis y GDS corresponden a `final_opt`.

¿Cuál es la diferencia entre KLayout y la interfaz de OpenROAD para ver el layout?

Ambos abren el GDS. KLayout permite superponer reportes de DRC y LVS y anotar; la interfaz de OpenROAD permite buscar instancias por nombre, útil para ubicar bloques jerárquicos.

10. Punto 3.7: Simulación post-síntesis

⋆ ¿Qué se simuló y qué se demostró?

El *netlist* post-síntesis (sin retardos anotados) con Verilator, usando modelos funcionales de las celdas de GF180 y un testbench que presiona botones en ciclos definidos (con rebote de pulsador) y juega 10 rondas, incluida una con botón incorrecto y otra con botón sostenido. Resultado: **trazas idénticas** entre RTL y netlist, `JUEGO: OK`, 0 fallos y 66 de 66 eventos verificados. Esto demuestra que la síntesis conserva el comportamiento y que una ronda correcta mide el mismo número de ciclos.

⋆ ¿Cómo comprobaron que una pulsación incorrecta reinicia la ronda?

El testbench aplica un botón distinto del LED objetivo y verifica que los displays muestren `EE` y que la ronda se repita; también verifica que apretar durante la fase de los 4 LEDs no cuenta como error. Se cumple igual en RTL y netlist.

⋆ ¿Por qué acortaron la espera de 3 s y cómo lo documentaron?

Con las constantes reales son $3 \cdot 10^7$ ciclos, demasiados para simular. Se sintetizó una variante (`sim_corto`) con constantes cortas (espera de 3 s = 20.000 ciclos, décima = 1.000, antirrebote = 200, parpadeo = 2.000) y se usó **la misma configuración en ambas simulaciones**, como exige el enunciado. El cambio está en el informe.

¿Por qué escribieron modelos propios de flip-flops?

Las celdas de flip-flop del PDK están definidas con primitivas de usuario (UDP) que Verilator no simula. Se escribieron modelos funcionales equivalentes para las 3 celdas (con y sin reset); las celdas combinacionales sí se tomaron del PDK, excluyendo las de tipo *hold* y fuerza débil.

¿Qué no cubre esta simulación?

Los retardos: es una simulación funcional sin SDF. La simulación post-PnR con retardos era opcional y no se hizo.

11. Preguntas conceptuales de apoyo

¿Qué diferencia hay entre FPGA y ASIC?

La FPGA es hardware reconfigurable: sus flip-flops arrancan en valores conocidos, trae bloques de memoria (BRAM) y se carga un archivo al encender. El ASIC es un circuito fijo hecho con celdas fabricadas: no hay carga de archivos, los flip-flops arrancan indeterminados y todo bloque (como una memoria) debe existir en el proceso o construirse con celdas.

¿Qué es una celda estándar?

Un bloque pequeño ya diseñado y caracterizado (compuerta, flip-flop, buffer) de altura fija, que se coloca en filas. El diseño digital se arma encadenando estas celdas.

¿Qué es un *latch-up*, un flip-flop y la diferencia con un *latch*?

El *latch-up* es un cortocircuito parásito entre alimentación y tierra del CMOS; se previene con las celdas *tap*. Un flip-flop captura el dato en el flanco del reloj; un *latch* es sensible al nivel del reloj. El diseño usa solo flip-flops.

¿Qué es la metaestabilidad?

Estado indefinido que puede tener un flip-flop cuando su entrada cambia cerca del flanco del reloj. Se mitiga con sincronizadores de dos flip-flops en entradas asíncronas (botones, reset).

¿Qué es el TNS y por qué importa que sea 0?

Es la suma de todos los slacks negativos. Que sea 0 significa que **ningún camino** falla, no solo el peor.

¿Qué es un *netlist* y qué es un LEF?

El *netlist* es la lista de celdas y conexiones que produce la síntesis. El LEF describe la *abstracción física* del macro (tamaño, pines, capas de metal) para poder usarlo en otro diseño.

¿Qué es RV32E y en qué difiere de RV32I?

Es el mismo conjunto de instrucciones de 32 bits pero con **16 registros** en lugar de 32, pensado para sistemas pequeños. Ahorra área en el register file, que aquí es el bloque más grande.

12. Limitaciones honestas: qué decir sin dudar

Si les preguntan por alguno de estos puntos, la respuesta correcta es reconocerlo con precisión.

Punto	Qué decir
Estrategia 2 incompleta	Se detuvo en el paso 43 con ruteo estancado en una violación; sus números son parciales. La conclusión (la ROM es la opción viable) no depende de cerrarla.
Barrido de reloj con RTL base	Solo el punto de 100 ns se corrió con el comparador de saltos. El resto del barrido es con el RTL anterior.
Frecuencia máxima extrapolada	65,1 ns y 62,8 ns se calcularon desde el slack de 100 ns; lo medido es 70 ns.
Causa del buffering invariante	Que el *resizer* repare solo en el corner típico es una hipótesis, no se verificó con otra corrida.
Slew y capacitancia	Quedan 2.426 avisos de *slew* en la corrida final; el experimento `final_slew` los reduce a 443 pero no reemplaza la corrida final.
DRC de KLayout omitido	El PDK `gf180mcuD` no trae reglas para KLayout; el DRC válido es el de Magic.
Costo lineal	Es una aproximación solo para este ejercicio.
Simulación sin retardos	Es funcional; no hay simulación con SDF.
Área mínima probada 60 %	Es el mínimo entre los puntos probados, no necesariamente el mínimo absoluto.
Uso de IA	Está declarado en el informe. Todos deben poder explicar el código y el flujo; no se puede responder "eso lo hizo la IA".

13. Dónde mirar cada tema (para estudiar o consultar)

Tema	Archivo o carpeta del repositorio
Módulo superior y reset	`rtl_asic/asic_top.v`
Register file ASIC	`rtl_asic/espino_core/espino_register_file.v`
Comparador de saltos	`rtl_asic/espino_core/espino_id_stage.v`
El juego y sus constantes	`sw/game.s`
Ensamblador y sus pruebas	`asm/pochoasm.py`
Configuración del flujo	`librelane/config.json`, `librelane/pin_order.cfg`, `librelane/variants/`
Evidencia por punto del enunciado	`informe/evidencia_3_*.md`
Pines y su justificación	`informe/pines_justificacion.md`
Métricas, reportes, GDS	`resultados/` (una carpeta por corrida)
Simulación post-síntesis	`sim_asic/`
Versiones	`versiones_herramientas.txt` y `README.md`

14. Si te preguntan algo que no sabes

- Decir qué sí se sabe y de dónde sale: "eso está en el reporte de síntesis" o "se midió en la corrida `final_opt`".
- Separar **lo medido** de **lo estimado**: la frecuencia máxima extrapolada y la explicación del *resizer* no son mediciones.
- Si es una pregunta de otro integrante, pedir un momento para consultarlo y volver con la respuesta; es mejor que improvisar un número.
- Reconocer límites con precisión es mejor evaluado que una respuesta inventada.